Audible Lava / DSO v4.13 — one blob, one voice: tracker stability companion

Sam Leventhal · November 2026 · companion to tracker_stability.py, test_tracker_stability.py and the v4.13 audible_lava.py

1. The symptom

A stationary blob whose detected persistence region fluctuates — growing, shrinking to a core, splitting into lobes, dropping out for a detection cycle — produced a stream of new voices, each with a birth strike, that glided up or down and away from the blob, instead of one voice that stayed with it. v4.12's voice continuity made sure every tracked blob *has* a voice; this release makes sure each physical blob is *one* track, so it gets one voice.

2. Why it happened

All four causes are in BlobTracker.update() (blob_tracker.py). IoU = intersection over union of two masks.

#	Cause	Mechanism
1	A change of shape counts as a new blob	Primary association is IoU ≥ 0.25 against each track's flow-advected mask. A region that shrinks to a core or changes to one lobe in place fails it; the detection is unmatched and, unless the re-identification buffer recognises it, _spawn() emits ('birth', id): a new voice and a strike.
2	Re-identification only covers dead tracks	The original track is not dead: it coasts for max_coast detection cycles. Nothing compares the 'new' detection with the coasting track it belongs to, so for that whole window there are two tracks for one blob, and later a death and a zombie voice.
3	Coasting extrapolates stale motion (the flying away)	kf.predict() runs on every frame: y += vy + 0.5·ay, vy += ay. Centroid jumps from shape changes read as velocity, and within 12 % of the top/bottom the measurement noise is raised from 0.05 to 2.0 so the buoyancy model is trusted — exactly where stationary pooled blobs sit. Once the detection is lost the predicted centroid keeps going for max_coast × K frames; pitch follows y_norm, so the voice glides away.
4	Fragments of a matched blob become births	Branch deduplication can keep a second H0 peak inside one wax mass for a few cycles (a peak whose merge partner was itself dropped), and the watershed then cuts the blob in two. The track matches one piece; the other is unmatched, and split detection only fires when the parent is coasting.

A fifth effect turned up while testing: when the region alternates, the base's flow-advected mask erodes (about 1,150 px against a real blob of about 1,900 px in the test), so it is a poor reference for 'where the blob is'. The fixes below keep their own reference — the **known extent**, the mask and area of the track's last *full* detection.

3. The fixes

tracker_stability.StableBlobTracker subclasses BlobTracker and wraps update() with a pre-pass and a post-pass. blob_tracker.py is not modified. Each fix has its own constructor switch.

Fix	Switch	What it does
F fuse fragments	fuse_fragments	Before association: detections that touch each other (within 2 px, so across a watershed line) and lie mostly (≥ absorb_frac = 0.5) inside the *same* track's mask are fused into one detection: union contour, summed area, area-weighted centroid, topology of the most persistent fragment, shape recomputed with BlobDetector's formulas.
P partial views	complete_partials	Before association: a *lone* detection inside one track's known extent at under partial_frac (0.5) of its known area is a partial view. It is replaced by a detection with the blob's known geometry and the detection's topology, whose centroid moves only partial_pull (15 %) of the way toward the core. Association then succeeds and neither the centroid nor the pitch jumps. A shrink that lasts partial_max_run (8) cycles is accepted as real.

Fix	Switch	What it does
R re-associate	reassociate	After association: each newly born track is offered to the tracks that missed this cycle, compared with their known extent and last measured centroid (not the extrapolated ones), by overlap coefficient $\|A \cap B\| / \min(\|A\|, \|B\|)$, which tolerates shape change. Overlap ≥ 0.3 decides on its own; otherwise the region must be within 1.5 radii and of similar size (×0.1 to ×4). Hungarian one-to-one assignment. A match folds the measurement into the old id and drops the birth: same voice, no strike.
T confirm births	confirm_cycles	A remaining new track is tentative: hidden from the returned list (so no voice and no v4.12 rebind) and its birth withheld until it has been detected on 3 consecutive cycles. A tentative track that misses once is dropped silently, with its re-ID record. One born inside a live track's mask confirms as ('split', parent, child). Deaths and merges of tentative tracks are silent.
D damp coasting	damp_coasting	While a track coasts, vx, vy and ay are multiplied by 0.7 every frame and the centroid is kept within one radius of the last measured position. A stationary track simply holds still.
S stationary mode	stationary_mode	A track whose smoothed speed (from an EMA, exponential moving average, of position, so shape jitter does not read as motion) is under 0.3 px/frame keeps the mid-column measurement noise (0.05) even near the top/bottom, has ay zeroed and its velocity halved on each detection.

4. Order of operations per detection cycle

```
StableBlobTracker.update(detections, frame, grey)
  F  _fuse(dets)                fragments of one tracked blob -> one detection
  P  _complete(dets)            lone partial view -> blob's known geometry, centroid pulled 15 %
     snapshot: tracks, tentative ids, pre-advection masks, stationary Kalman filters
     BlobTracker.update(dets)   unchanged: flow, advection, IoU match, merge, coast, death, split, re-ID, spawn
                                (stationary filters get 0.05 measurement noise for this call only)
  R  _reassociate(new births)   -> coasting tracks, Hungarian, fold + drop birth
  T  _gate_births / _settle     new births become tentative; confirm after 3; drop on a miss
     _bookkeep_measured         last measured centroid, speed EMA, known extent (full views only)
  D  _constrain_coasting        decay velocity, clamp drift, hold stationary tracks
     return get_active()        confirmed tracks only

Predict-only frames: BlobTracker.update(None) then D.
```

The stationary noise is applied by swapping blob_tracker._adapt_measurement_noise for the duration of one update() call; it is restored in a finally block, and a test checks the module function afterwards.

5. Results

Synthetic sequences rendered as grey frames, so the real Farneback flow and the real buoyancy Kalman filter run; detections are built from masks with BlobDetector's shape formulas. Same input to both trackers.

Scenario	BlobTracker (v4.12)	StableBlobTracker (v4.13)
Still blob near the top, region alternating whole / core / lobe / lobe, 40 cycles	7 births, 7 ids; centroid spread 135 × 33 px	1 birth, 1 id; centroid spread 5.6 × 0.5 px
Jittered blob near the top, then 6 missed cycles at K = 3 (coasting)	centroid drifts 57 px (radius 22 px)	10 px
Blob cut in two by a watershed line for 6 cycles	2 births	1 birth; centroid stays on the whole blob
Blob vanishes 3 cycles, returns with another shape	—	1 birth, same id
Real blob appears; another flickers for one cycle	—	real one born on its 3rd detection; flicker never visible
Blob rising 1.5 px/frame (also alternating with a core view)	—	1 birth, RISING, vy ≈ –1.5, on course
Two fluctuating blobs 70 px apart	—	2 ids, never swapped or fused
Disjoint lobe persisting inside a blob's mask	—	('split', parent, child) after 3 cycles

Scenario	BlobTracker (v4.12)	StableBlobTracker (v4.13)
Genuine shrink to a third	—	accepted after 8 cycles; 1 birth
All switches off	—	event stream, ids and centroids identical to BlobTracker

test_tracker_stability.py: 14 tests (no camera, SuperCollider or pygame) — **14 passed**. Cost with six fluctuating blobs in a 360 × 640 ROI (region of interest): BlobTracker 28.9 ms per detection cycle, StableBlobTracker 29.9 ms (bounding-box pre-checks and a per-cycle mask cache keep the extra work to about 1 ms). Predict-only frames add only the coasting clamp.

6. Settings

Argument	Default	Raise it to …	Lower it to …
confirm_cycles	3	ignore longer flickers (birth latency grows: 3 cycles ≈ 0.4 s at K = 2, 15 fps)	hear births sooner (1 = v4.12 behaviour)
absorb_frac	0.5	fuse / treat as part of a blob only when more clearly inside	absorb more readily
partial_frac	0.5	treat larger shrinkages as partial views	only treat tiny cores as partial
partial_pull	0.15	let partial views move the centroid more (better for moving blobs seen in part)	hold the centroid steadier
partial_max_run	8	hold the known size longer through flicker	accept real shrinkage sooner
reassoc_overlap / reassoc_gate	0.3 / 1.5	stricter / looser re-association	
coast_damping	0.7 /frame	let coasting tracks keep more momentum	stop them faster
coast_radius_frac	1.0	allow more coasting drift	pin coasting tracks closer
stationary_speed	0.3 px/frame	treat slow movers as still too	only hold truly still blobs

audible_lava.py exposes STABLE_TRACKER, STABLE_CONFIRM_CYCLES and STABLE_VERBOSE (print each fuse, re-association and dropped flicker); the rest are constructor arguments.

7. Trade-offs and limits

• **Birth latency.** A real new blob is heard 3 detection cycles later than before (about 0.4 s). Splits detected by the base are not delayed.

• **Partial views can mask a fast real shrink** for up to partial_max_run cycles (the blob keeps its known size and shape for its voice). Wax rarely halves in under a second, but tune it if it does.

• **Partially seen moving blobs lag.** A partial view moves the centroid only 15 % toward the core, so the full views carry the motion. In the test the blob stays on course; with long runs of partial views it would trail.

• **Fusion trusts the track mask.** Two genuinely separate blobs that touch inside one stale track mask would be fused. Blobs that already have their own tracks are never fused (ts_07).

• **Ambiguity is left to the base.** Completion only acts on a lone detection inside exactly one known extent; two non-touching detections inside one blob go through the tentative path and, if they persist, confirm as a split.

• **Synthetic evidence.** The numbers above are from rendered sequences, not lamp footage. Before trusting them, run the lamp with and without --no-stable-tracker and compare the HUD (heads-up display) line.

8. Wiring and checking it on the lamp

File	Status	Role
tracker_stability.py	NEW	StableBlobTracker (fixes F, P, R, T, D, S), hud_line(), stats, is_tentative()

File	Status	Role
test_tracker_stability.py	NEW	14 tests; stubs the GUDHI helpers if they are not importable
audible_lava.py	MODIFIED (additive)	imports tracker_stability; builds StableBlobTracker with the same arguments; --no-stable-tracker; startup line; tracker HUD line appended to the first status line; window title v4.13
build_companion_v413.py	NEW	this document

HUD line: *trk births:N reassoc:N fused:N partial:N tentative:N dropped:N*. On a lamp with stationary blobs, births should now grow roughly with the number of real blobs, while reassoc / partial / dropped absorb the fluctuation. For an A/B comparison: *python audible_lava.py --no-stable-tracker* and watch the console '+ Voice birth' lines. Set STABLE_VERBOSE = True to print each intervention.